

📑 BÀI THỰC HÀNH FLYWEIGHT PATTERN

- ✓ Backend: ASP.NET Core Web API
- ✓ Frontend: React (gọi API, hiển thị danh sách Order Items)
- ✓ Áp dụng Flyweight Pattern đúng chuẩn GoF
- ✓ Minh họa rõ việc chia sẻ dữ liệu dùng chung để tối ưu bộ nhớ

⚙️ BÀI TOÁN

Hệ thống **Order Management** trong E-commerce có rất nhiều đơn hàng. Mỗi đơn hàng chứa nhiều **OrderItem**, nhưng nhiều item có thể tham chiếu đến cùng một sản phẩm.

Ví dụ:

- Order 1 chứa Laptop
- Order 2 cũng chứa Laptop
- Order 3 cũng chứa Laptop

Nếu mỗi `OrderItem` đều lưu toàn bộ thông tin sản phẩm như:

- Tên sản phẩm
- Giá
- Danh mục
- Mô tả

thì sẽ gây **lãng phí bộ nhớ** khi hệ thống có hàng nghìn đơn hàng.

👉 Giải pháp: dùng **Flyweight Pattern** để chia sẻ thông tin sản phẩm dùng chung.

🏗️ KIẾN TRÚC HỆ THỐNG

```
Frontend (React)
    ↓
GET /api/orders/items
    ↓
OrderService
    ↓
ProductFactory (Flyweight Factory)
    ↓
ProductFlyweight
    ↓
```

OrderItem

BACKEND – ASP.NET Core 8 Web API

1 Tạo project

```
dotnet new webapi -n OrderFlyweightDemo
cd OrderFlyweightDemo
```

Cấu trúc thư mục

```
OrderFlyweightDemo
├── Controllers
│   └── OrdersController.cs
├── Services
│   └── OrderService.cs
├── Flyweights
│   ├── ProductFlyweight.cs
│   ├── IProductFactory.cs
│   └── ProductFactory.cs
├── Models
│   ├── OrderItem.cs
│   ├── OrderItemDto.cs
│   └── ProductRequest.cs
└── Program.cs
```

1. FLYWEIGHT OBJECT

Flyweights/ProductFlyweight.cs

```
namespace OrderFlyweightDemo.Flyweights;

public class ProductFlyweight
{
    public string ProductId { get; }
    public string Name { get; }
    public decimal Price { get; }
    public string Category { get; }

    public ProductFlyweight(string productId, string
name, decimal price, string category)
    {
        ProductId = productId;
        Name = name;
    }
}
```

```
        Price = price;
        Category = category;
    }
}
```

👉 Đây là **shared object** chứa dữ liệu nội tại (**intrinsic state**) dùng chung giữa nhiều OrderItem.

✿ 2. FLYWEIGHT FACTORY INTERFACE

Flyweights/IProductFactory.cs

```
namespace OrderFlyweightDemo.Flyweights;

public interface IProductFactory
{
    ProductFlyweight GetProduct(string productId, string
name, decimal price, string category);
    int GetTotalFlyweights();
}
```

✿ 3. FLYWEIGHT FACTORY

Flyweights/ProductFactory.cs

```
namespace OrderFlyweightDemo.Flyweights;

public class ProductFactory : IProductFactory
{
    private readonly Dictionary<string, ProductFlyweight>
_products = new();

    public ProductFlyweight GetProduct(string productId,
string name, decimal price, string category)
    {
        if (!_products.ContainsKey(productId))
        {
            _products[productId] = new
ProductFlyweight(productId, name, price, category);
        }

        return _products[productId];
    }
}
```

```
        public int GetTotalFlyweights()
        {
            return _products.Count;
        }
    }
```

👉 Factory đảm bảo mỗi sản phẩm chỉ có **một object duy nhất trong memory**.

🌀 4. CONTEXT OBJECT

Models/OrderItem.cs

```
using OrderFlyweightDemo.Flyweights;

namespace OrderFlyweightDemo.Models;

public class OrderItem
{
    public int OrderId { get; set; }
    public ProductFlyweight Product { get; set; }
    public int Quantity { get; set; }

    public decimal GetTotal()
    {
        return Product.Price * Quantity;
    }
}
```

👉 OrderItem là **context object**:

- Chứa dữ liệu riêng (**extrinsic state**) như OrderId, Quantity
 - Dùng chung ProductFlyweight
-

🌀 5. DTO MODEL

Models/OrderItemDto.cs

```
namespace OrderFlyweightDemo.Models;

public class OrderItemDto
{
    public int OrderId { get; set; }
```

```
    public string ProductId { get; set; }
    public string ProductName { get; set; }
    public string Category { get; set; }
    public decimal Price { get; set; }
    public int Quantity { get; set; }
    public decimal Total { get; set; }
}
```

6. SERVICE

Services/OrderService.cs

```
using OrderFlyweightDemo.Flyweights;
using OrderFlyweightDemo.Models;

namespace OrderFlyweightDemo.Services;

public class OrderService
{
    private readonly IProductFactory _productFactory;

    public OrderService(IProductFactory productFactory)
    {
        _productFactory = productFactory;
    }

    public List<OrderItem> BuildOrderItems()
    {
        var items = new List<OrderItem>();

        var laptop = _productFactory.GetProduct("P01",
            "Laptop", 1500, "Electronics");
        var mouse = _productFactory.GetProduct("P02",
            "Mouse", 20, "Accessories");
        var laptopAgain =
            _productFactory.GetProduct("P01", "Laptop", 1500,
            "Electronics");

        items.Add(new OrderItem
        {
            OrderId = 1001,
            Product = laptop,
            Quantity = 1
        });

        items.Add(new OrderItem
```

```

        {
            OrderId = 1002,
            Product = mouse,
            Quantity = 3
        });

        items.Add(new OrderItem
        {
            OrderId = 1003,
            Product = laptopAgain,
            Quantity = 2
        });

        return items;
    }

    public List<OrderItemDto> GetOrderItemDtos()
    {
        return BuildOrderItems()
            .Select(item => new OrderItemDto
            {
                OrderId = item.OrderId,
                ProductId = item.Product.ProductId,
                ProductName = item.Product.Name,
                Category = item.Product.Category,
                Price = item.Product.Price,
                Quantity = item.Quantity,
                Total = item.GetTotal()
            })
            .ToList();
    }

    public object GetSummary()
    {
        var items = BuildOrderItems();

        return new
        {
            totalOrderItems = items.Count,
            totalFlyweights =
                _productFactory.GetTotalFlyweights(),
            grandTotal = items.Sum(i => i.GetTotal())
        };
    }
}

```

7. CONTROLLER

Controllers/OrdersController.cs

```
using Microsoft.AspNetCore.Mvc;
using OrderFlyweightDemo.Services;

namespace OrderFlyweightDemo.Controllers;

[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly OrderService _orderService;

    public OrdersController(OrderService orderService)
    {
        _orderService = orderService;
    }

    [HttpGet("items")]
    public IActionResult GetItems()
    {
        var items = _orderService.GetOrderItemDtos();
        return Ok(items);
    }

    [HttpGet("summary")]
    public IActionResult GetSummary()
    {
        var summary = _orderService.GetSummary();
        return Ok(summary);
    }
}
```

8. ĐĂNG KÝ DI

Program.cs

```
using OrderFlyweightDemo.Flyweights;
using OrderFlyweightDemo.Services;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
```

```
builder.Services.AddSingleton<IProductFactory,
ProductFactory>();
builder.Services.AddTransient<OrderService>();

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowReact", policy =>
    {
        policy.WithOrigins("http://localhost:3000")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseCors("AllowReact");
app.MapControllers();

app.Run();
```

CHẠY BACKEND

dotnet run

Swagger:

<https://localhost:xxxx/swagger>

Test API:

GET /api/orders/items
GET /api/orders/summary

FRONTEND – REACT

Tạo project

```
npx create-react-app order-flyweight-ui
cd order-flyweight-ui
npm install axios
```

src/App.js

```
import React, { useEffect, useState } from "react";
import axios from "axios";

function App() {
  const [items, setItems] = useState([]);
  const [summary, setSummary] = useState(null);

  useEffect(() => {
    axios.get("https://localhost:xxxx/api/orders/items")
      .then(res => setItems(res.data));
  });

  axios.get("https://localhost:xxxx/api/orders/summary")
    .then(res => setSummary(res.data));
  }, []);

  return (
    <div style={{ padding: 40 }}>
      <h2>Order Items</h2>

      <table border="1" cellPadding="10" style={{
borderCollapse: "collapse" }}>
        <thead>
          <tr>
            <th>Order ID</th>
            <th>Product</th>
            <th>Category</th>
            <th>Price</th>
            <th>Quantity</th>
            <th>Total</th>
          </tr>
        </thead>
        <tbody>
          {items.map((item, index) => (
            <tr key={index}>
              <td>{item.orderId}</td>
              <td>{item.productName}</td>
              <td>{item.category}</td>
```

```

                <td>{item.price}</td>
                <td>{item.quantity}</td>
                <td>{item.total}</td>
            </tr>
        )}
    </tbody>
</table>

    {summary && (
        <div style={{ marginTop: 30 }}>
            <h3>Summary</h3>
            <p>Total Order Items:
{summary.totalOrderItems}</p>
            <p>Total Flyweights:
{summary.totalFlyweights}</p>
            <p>Grand Total: {summary.grandTotal}</p>
        </div>
    )}
</div>
);
}

export default App;

```

▶ CHẠY FRONTEND

```
npm start
```

🔗 LUỒNG XỬ LÝ

Khi gọi `/api/orders/items`

1. Controller gọi `OrderService.GetOrderItemDtos()`
 2. Service gọi `BuildOrderItems()`
 3. `ProductFactory` kiểm tra sản phẩm đã tồn tại chưa
 4. Nếu chưa có → tạo mới `ProductFlyweight`
 5. Nếu đã có → dùng lại object cũ
 6. Trả về danh sách `OrderItemDto`
-

Khi gọi `/api/orders/summary`

- Tính:

```
items.Count  
_productFactory.GetTotalFlyweights()  
items.Sum(i => i.GetTotal())
```

🔗 Kết quả cho thấy:

- Có nhiều `OrderItem`
 - Nhưng số `Flyweight` object ít hơn
 - Chứng minh được việc **chia sẻ object để tiết kiệm bộ nhớ**
-

🔗 KIẾN THỨC SINH VIÊN ĐẠT ĐƯỢC

✓ Hiểu Flyweight Pattern (GoF)

✓ Phân biệt:

- **Intrinsic state:** dữ liệu dùng chung trong `ProductFlyweight`
- **Extrinsic state:** dữ liệu riêng trong `OrderItem`

✓ Biết cách:

- Dùng Factory để quản lý object dùng chung
- Tối ưu memory trong hệ thống có số lượng object lớn

✓ Áp dụng thực tế:

- Product catalog
 - Order management
 - Inventory system
-

🚀 BÀI TẬP MỞ RỘNG

1. Thêm API:

- Tạo đơn hàng mới
- Thêm nhiều sản phẩm vào order

2. Mở rộng Flyweight:

- Thêm `Brand`, `Description`, `ImageUrl`

3. Áp dụng vào:

- Cart system
 - Invoice system
 - Warehouse management
-

🌀 TÓM TẮT Ý NGHĨA PATTERN

Flyweight Pattern giúp:

- Giảm số lượng object tạo ra
- Tối ưu bộ nhớ
- Tăng hiệu năng trong hệ thống có nhiều dữ liệu lặp lại

Trong bài này:

- `ProductFlyweight` là dữ liệu chia sẻ
- `OrderItem` là ngữ cảnh sử dụng dữ liệu đó
- `ProductFactory` chịu trách nhiệm cache và tái sử dụng object